

CSE232 시스템프로그래밍(캡스톤디자인)

3주차 첫 번째 수업

저수준 파일 입출력 파일 프로그램 4개

경민기

수업 구성

1부 지난 시간 복습 · 10분

2부 파일 이해와 생성 · 25분

3부 파일 쓰기 · 20분

4부 파일 읽기 · 20분

5부 지정 위치 수정 · 25분

3부 파일 쓰기 · 20분

1부

지난 시간 복습

컴파일 · Makefile · 리다이렉션과 종료 상태를 복습합니다.

컴파일과 Makefile

```
cc -Wall -Wextra -g helloworld.c -o helloworld
./helloworld
make
```

소스를 수정하면 다시 빌드합니다.

컴파일

소스 파일과 실행 파일을 구별한다.

Makefile

make는 Makefile의 규칙을 실행한다.

재빌드

소스를 수정하면 다시 빌드한다.

표준 출력과 종료 상태

```
./helloworld > hello.txt  
./helloworld >> hello.txt  
cat hello.txt  
echo $?
```

덮어쓰기

>는 파일을 새로 만들거나 비운다.

추가 기록

>>는 파일 끝에 추가한다.

종료 상태

\$?는 직전 명령의 종료 상태다.

\$?는 바로 앞 명령의 종료 상태입니다.

2부

파일 이해와 생성

파일 구조를 이해하고 `file_create`로 빈 파일을 만듭니다.

파일이란 무엇인가

일반 파일은 프로그램이 데이터를 담아 두는 저장 단위다.

프로그램을 종료해도 저장한 내용은 다시 읽을 수 있다.

내용은 순서가 있는 바이트이며 크기는 바이트 수다.

텍스트·이미지·실행 파일은 내용을 해석하는 방식이 다르다.

파일의 내용은 바이트의 나열

```
printf 'ABC' > sample.txt  
wc -c < sample.txt  
od -An -tx1 sample.txt
```

개행과 NUL도 각각 하나의 바이트입니다.

바이트

텍스트 파일도 바이트로 저장한다.

개행과 NUL

개행도 저장 공간을 차지한다.
C 문자열 끝의 NUL은 자동 저장되지 않는다.

한글

한글의 문자 수와 바이트 수는 다를 수 있다.

파일 내용과 메타데이터

내용: 파일에 저장된 실제 바이트

메타데이터: 종류·크기·소유자·권한·시간 정보

inode: 파일의 메타데이터를 관리하는 구조

cat은 내용을, ls -l과 stat은 정보를 보여 준다.

파일 이름, 디렉터리와 경로

디렉터리는 파일 이름과 inode 번호의 연결을 관리한다.

경로는 디렉터리를 따라 파일을 찾는 방법이다.

절대 경로: `/home/tester/class/note.txt`

상대 경로: 현재 작업 디렉터리 기준의 `note.txt`

Linux에서 만나는 파일 종류

- 일반 파일: 텍스트·이미지·실행 파일

d 디렉터리: 이름과 파일의 연결 관리

l 심볼릭 링크: 다른 경로를 가리키는 링크

c / b 장치 파일 p / s FIFO·소켓

관찰 · 내용과 파일 정보 비교

```
DEMO=$(mktemp -d)
printf 'ABC\n' > "$DEMO/note.txt"
cat "$DEMO/note.txt"
wc -c < "$DEMO/note.txt"
od -An -tx1 "$DEMO/note.txt"
ls -l "$DEMO/note.txt"
stat "$DEMO/note.txt"
```

내용

cat으로 ABC를 읽는다.
끝에 개행도 들어 있다.

바이트

크기는 4바이트다.
16진수: 41 42 43 0a

정보

ls -l과 stat에서 크기·권한·
소유자 정보를 확인한다.

개행도 파일 내용에 포함됩니다. 권한과 소유자는 실행 환경에 따라 다릅니다.

경로, FD, 열린 파일 상태

경로: open에 전달하는 파일 이름

FD: 프로세스가 사용하는 0 이상의 정수

열린 파일 상태: 현재 오프셋과 상태 flags

read/write는 경로 대신 FD를 사용한다

표준 입력 · 출력 · 오류

0 STDIN_FILENO 표준 입력

1 STDOUT_FILENO 표준 출력

2 STDERR_FILENO 표준 오류

보통 터미널에 연결되지만 리다이렉션할 수 있다.

새 FD가 반드시 3이라는 보장은 없다.

open과 close

```
int fd = open("sample.txt", O_RDONLY);
if (fd == -1) {
    perror("open");
    return 1;
}
/* fd를 사용하는 작업 */
if (close(fd) == -1) {
    perror("close");
    return 1;
}
```

FD 0도 유효하며 실패는 -1입니다.

반환값

성공한 open은 FD를 반환한다.

오류 확인

실패는 -1이며 perror로 이유를 출력한다.

자원 반환

close는 성공 0, 실패 -1이다.
사용을 끝낸 FD는 닫는다.

생성 권한과 umask

```
int fd = open("note.txt",  
    O_WRONLY | O_CREAT | O_EXCL,  
    0666);
```

기본 ACL이 없는 디렉터리에서 실행합니다.

생성 권한

0666은 rw-rw-rw-를 요청한다.

umask 적용

기본 ACL이 없을 때 $\text{mode} \& \sim \text{umask}$

022: 0644 / 027: 0640

기존 파일

mode는 이미 존재하는 파일의 권한을 바꾸지 않는다.

file_create의 핵심 코드

```
int fd = open(argv[1],
    O_WRONLY | O_CREAT | O_EXCL, 0666);
if (fd == -1) {
    perror(argv[1]); return 1;
}
if (close(fd) == -1) {
    perror("close"); return 1;
}
```

완성 코드는 file_create.c에 있습니다.

생성 정책

O_CREAT와 O_EXCL을 함께 사용한다.

권한

생성 권한은 0666을 요청한다.

빈 파일

기록하지 않고 닫으면 빈 파일이 남는다.

실습 1 · 빈 파일 만들기

```
make
RUN=$(mktemp -d ./run.XXXXXXX)
./file_create "$RUN/note.txt"
wc -c < "$RUN/note.txt"
./file_create "$RUN/note.txt"
echo $?
ls -l "$RUN/note.txt"
```

같은 셸에서 RUN 변수를 이어서 사용합니다.

실행

같은 파일을 두 번 생성한다.

오류

두 번째 실행은 종료 상태 1이다.

관찰

파일 크기와 권한도 확인한다.

3부

파일 쓰기

file_write로 덮어쓰기와 추가 기록을 비교합니다.

접근 모드와 추가 flags

접근 모드는 아래 중 하나를 선택한다.

O_RDONLY: 읽기

O_WRONLY: 쓰기

O_RDWR: 읽기와 쓰기

추가 flags는 비트 OR 연산자 |로 결합한다.

생성과 기록 방식

O_CREAT: 파일이 없으면 생성

O_EXCL + O_CREAT: 이미 있으면 실패

O_TRUNC: 쓰기용으로 연 일반 파일의 길이를 0으로

O_APPEND: 매 write를 파일 끝에 기록

write와 부분 쓰기

write의 결과는 실제 기록한 바이트 수다.

요청한 길이보다 적게 기록할 수 있다.

남은 위치와 길이를 갱신해 다시 쓴다.

이번에는 제공한 write_all을 사용한다.

다음 시간에는 반복문을 직접 구현한다.

파일 기록기의 핵심

```
int fd = open(path, flags, 0666);
if (fd == -1) { perror(path); return 1; }
int status = 0;
if (write_all(fd, text, strlen(text)) == -1) {
    perror("write"); status = 1;
}
if (close(fd) == -1) {
    perror("close"); status = 1;
}
return status;
```

완성 코드는 file_write.c에 있습니다.

기록 방식

flags가 기록 방식을 결정한다.

문자열 길이

strlen은 끝의 NUL을 제외한다.

오류 처리

쓰기 실패 뒤에도 FD를 닫는다.

실습 2 · 덮어쓰기와 추가

```
./file_write truncate "$RUN/note.txt" ABCDEFGHIJ
./file_write truncate "$RUN/note.txt" XY
./file_write append "$RUN/note.txt" ZZ
cat "$RUN/note.txt"; printf '\n'
wc -c < "$RUN/note.txt"
```

덮어쓰기

truncate는 원래 내용을 비운다.

추가

append는 기존 내용을 유지한다.

결과 확인

최종 내용과 바이트 수를 예상한다.

마지막 파일 내용은 XYZZ, 크기는 4바이트입니다.

4부

파일 읽기

file_read로 EOF까지 읽고 반환값을 확인합니다.

read의 반환값

```
unsigned char buf[16];  
ssize_t n = read(fd, buf, sizeof buf);
```

양수와 EOF

$n > 0$: 실제로 읽은 바이트 수

$n == 0$: 일반 파일의 EOF

오류

$n == -1$: 오류

반복 읽기

요청보다 적게 읽어도 다음 read를 계속한다.

일반 파일에서 $\text{count} > 0$ 인 read를 기준으로 설명합니다.

읽은 바이트만 사용하기

```
/* n > 0인 경우 */  
write_all(STDOUT_FILENO, buf, (size_t)n);  
  
/* 피해야 할 가정 */  
printf("%s", buf);  
/* NUL 종료가 보장되지 않는다. */
```

실제로 읽은 n바이트만 출력합니다.

문자열 종료

read는 문자열 끝에 NUL을 붙이지 않는다.

유효한 길이

버퍼 크기와 실제 읽은 길이를 구별한다.

바이너리

바이너리 파일에도 같은 방식으로 적용한다.

EOF까지 반복 읽기

```
for (;;) {
    ssize_t n = read(fd, buf, sizeof buf);
    if (n == 0) break;
    if (n == -1) {
        if (errno == EINTR) continue;
        perror("read"); status = 1; break;
    }
    if (write_all(1, buf, (size_t)n) == -1) {
        perror("write"); status = 1; break;
    }
}
```

쓰기 실패 뒤에도 획득한 FD를 닫습니다.

종료 구분

오류와 EOF는 서로 다른 종료 경로다.

EINTR

EINTR는 읽기 중 신호로 중단된 경우다.

재시도

완성 소스는 EINTR 때 다시 읽는다.

실습 3 · 버퍼와 EOF

```
printf '0123456789abcdefghijklmnopqrstuv' > "$RUN/long.txt"
./file_read "$RUN/long.txt" > "$RUN/copy.txt"
cmp "$RUN/long.txt" "$RUN/copy.txt"
./file_create "$RUN/empty.txt"
./file_read "$RUN/empty.txt"; echo $?
./file_read "$RUN/missing.txt"; echo $?
```

버퍼

16바이트 버퍼로 32바이트 파일을 읽는다.

빈 파일과 오류

빈 파일과 없는 파일도 비교한다.

크기 변경

버퍼를 7바이트로 바꿔 결과를 관찰한다.

자세한 수정 과제는 실습 안내를 참고합니다.

5부

지정 위치 수정

file_seek로 지정 위치를 읽고 일부 바이트를 수정합니다.

lseek와 세 가지 기준

```
off_t pos = lseek(fd, 3, SEEK_SET);  
if (pos == (off_t)-1) {  
    perror("lseek");  
}
```

SEEK_SET의 첫 바이트 위치는 0입니다.

위치 기준

SEEK_SET: 파일 시작 기준
SEEK_CUR: 현재 위치 기준
SEEK_END: 파일 끝 기준

반환값

성공하면 새 위치, 실패하면 (off_t)-1

단위

위치의 단위는 바이트이며 첫 위치는 0이다.

오프셋 3에 XY를 쓰면

변경 전: ABCDEFGHIJ

위치: 0123456789

변경 후: ABCXYFGHIJ

D와 E를 바꾸며 파일 크기는 10바이트다.

한글은 여러 바이트일 수 있으므로 ASCII로 실습한다.

읽은 뒤에는 위치가 이동한다

```
lseek(fd, offset, SEEK_SET);  
n = read(fd, buf, sizeof buf);  
  
/* 원래 수정할 위치로 복귀 */  
lseek(fd, offset, SEEK_SET);  
write_all(fd, text, strlen(text));
```

읽기 후 위치

읽은 바이트 수만큼 위치가 이동한다.

쓰기 전 복귀

수정 전에 다시 위치 3으로 이동한다.

열기 모드

O_RDWR로 읽기와 쓰기를 허용한다.
O_APPEND와 O_TRUNC는
사용하지 않는다.

흐름 발췌입니다. 완성 코드에서는 모든 반환값을 검사합니다.

실습 4 · 지정 위치 수정

```
./file_write truncate "$RUN/note.txt" ABCDEFGHIJ
./file_seek "$RUN/note.txt" 3 XY
printf '\n'
./file_read "$RUN/note.txt"; printf '\n'
wc -c < "$RUN/note.txt"
```

수정 결과는 ABCXYFGHIJ, 크기는 10바이트입니다.

수정 전

수정 전 일부 내용을 표준 출력에 표시한다.

수정 후

다시 파일 전체를 읽어 결과를 확인한다.

위치 변경

오프셋 0과 파일 끝 위치도 실험한다.

위치 이동과 파일 크기

lseek만으로 파일 크기가 늘어나지는 않는다.

파일 끝에서 쓰면 길이가 늘어난다.

끝 너머에서 쓰면 사이 영역은 읽을 때 0이다.

파이프처럼 위치 이동이 불가능한 대상도 있다.

확인 퀴즈

1. open의 실패 반환값은?
2. 빈 파일과 없는 파일은 어떻게 구별하는가?
3. O_TRUNC와 O_APPEND의 차이는?
4. lseek(fd, 0, SEEK_CUR)는 무엇을 반환하는가?
5. read 후 같은 위치에 쓰려면 무엇이 필요한가?

제출과 다음 시간

제출: C 프로그램 4개와 REPORT.md

생성·쓰기·읽기·지정 위치 수정 결과 기록

수정 과제: 버퍼 크기와 위치 이동 기준 변경

다음 시간: 부분 쓰기 반복문과 파일 복사기